Práctica 6 – Backtracking V2

1)CUADRADO NUMÉRICO

a) Implementar en Java dicho algoritmo (CuadradoNumerico.java) de tal forma que calcule para una entrada de tablero dada una solución de la forma más eficiente posible.

b) Implementar en Java dicho algoritmo (CuadradoNumericoTodas.java) de tal forma que calcule para una entrada de tablero dada todas las soluciones posibles.

c) Razonar la Complejidad Temporal. ¿Cree que se podría resolver este problema de forma óptima utilizando algunas de las técnicas anteriores vistas durante el curso?

Caso de prueba	Tiempo para la primera solución (ms)	Tiempo para todas las soluciones (ms)	Número de soluciones encontradas
Test00	FdF	FdF	1
Test01	FdF	62	12
Test02	FdF	FdF	1
Test03	479	738	3
Test04	5561	7493	2
Test05	493	5403	5
Test06	110	51526	83
Test07	13071	FdT	-

Esto medido con un procesador Intel Core i5 12400 2.5GHz

Al tratarse de un caso de backtracking, sabemos que la complejidad de este viene dada por el número de nodos generados, así como el tiempo invertido en cada uno de estos.

En nuestro caso, al rellenar cada casilla un total de diez veces (de 0 a 9), la complejidad sería del número total a casillas por rellenar (las ‘?’) multiplicado por esto.

Por ende, es evidente que a mayor número de interrogantes (que no es lo mismo que mayor tamaño de matriz, aunque pueden ir de la mano) mayores serán los tiempos obtenidos.

Ya están resuelto los inconvenientes con los tests que comenté anteriormente, debiéndose estos a que yo daba por válidas los resultados de divisiones no enteras, por lo que en ciertos casos obtenía un número de soluciones mayor a las esperadas e, incluso, una primera solución que no era acorde a la puesta en el test.

Esto ya se encuentra resuelto, ahora ya sí pasando todos los test correctamente.

Como en ciertos casos el número de soluciones variaba considerablemente respecto a lo esperado, he rehecho la tabla de mediciones con esta versión a fin de ser lo más correcto posible.